Artificial Intelligence

AIML320/AIML420

Preprocessing

Dr. Heitor Murilo Gomes
School of Engineering and Computer Science
Adaptive AI Lab
heitor.gomes@vuw.ac.nz
http://www.heitorgomes.com

Outline

- Imbalanced Classification
  - Undersampling, oversampling and cost-sensitive
- Dimensionality Reduction
  - Principal Component Analysis (PCA)

Imbalanced classification

Imbalanced classification

- In many real classification problems, one class is much rarer than the others (e.g. spam detection, medical diagnosis)

- **Accuracy** alone can be misleading in these scenarios (see the evaluation metrics from earlier)

<u>Learning algorithms often favour the majority class</u>

Imbalanced classification

<u>Learning algorithms often favour the majority class</u>

- If the model is already correctly classifying the majority of examples, the remaining minority cases may contribute little to the optimisation objective

- Consequently, the learner may not prioritise learning additional “rules” to correctly classify those rare instances

Imbalanced classification

<u>Learning algorithms often favour the majority class</u>

- If the model is already correctly classifying the majority of examples, the remaining minority cases may contribute little to the <u>optimisation objective</u>

- Consequently, the learner may not prioritise learning additional “rules” to correctly classify those rare instances

Example

Suppose a decision tree with just one node (the root)

Class 0: 10 examples

Class 1: 100,000 examples

Imbalanced classification

<u>Learning algorithms often favour the majority class</u>

- If the model is already correctly classifying the majority of examples, the remaining minority cases may contribute little to the <u>optimisation objective</u>
- Consequently, the learner may not prioritise learning additional “rules” to correctly classify those rare instances

Example

Suppose a decision tree with just one node (the root)

Class 0: 10 examples

Class 1: 100,000 examples

This node is already **<u>almost pure</u>**

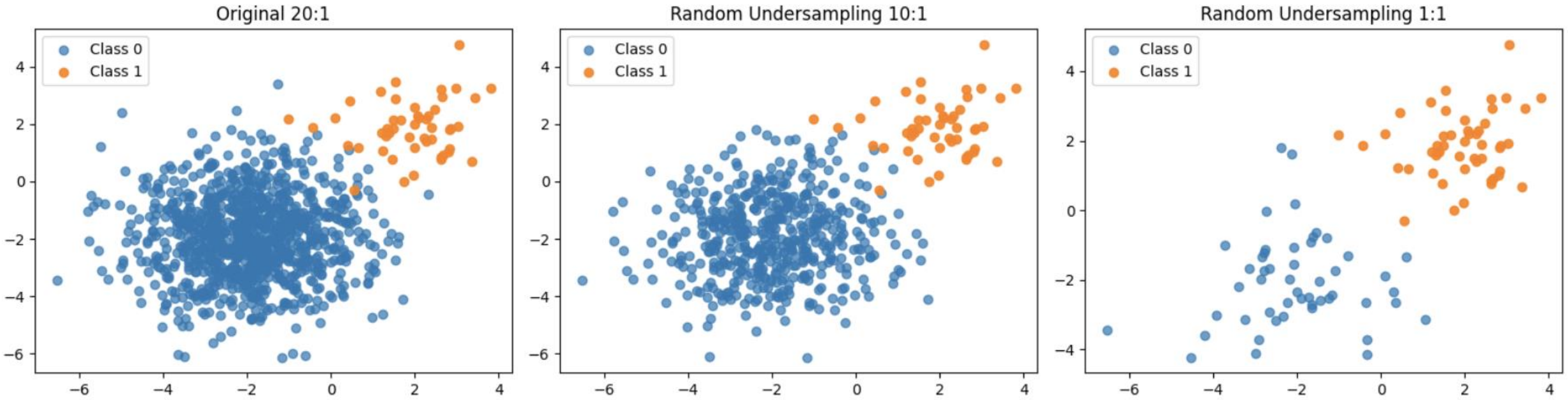
Why split further?
Any split that isolates the minority class will only produce a **very small reduction in impurity**, so depending on the stopping criteria the tree will simply not grow

Imbalanced classification

- What can we do to address class imbalance?
  - **Undersampling** the majority class
    - Remove instances from the majority
  - **Oversampling** the minority class
    - “Create” instances for the minority
  - **Cost-sensitive learning** (i.e.. class weighting)
    - Modify the <u>optimisation objective</u> to penalise mistakes on the minority class more heavily

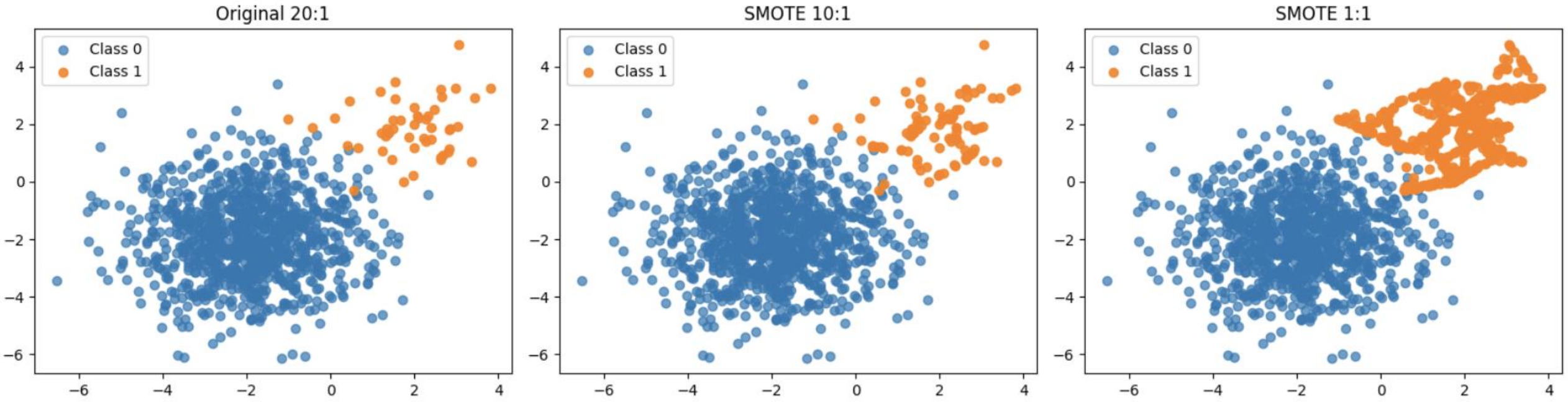
Undersampling

- Randomly remove instances from the majority class
- Reduces or eliminates class imbalance in the training dataset
- May discard useful information, since many majority examples are removed



Oversampling

- Add instances to the minority class to reduce class imbalance
- Makes minority cases appear more frequently during training
- Synthetic approaches (e.g. SMOTE[1]) create new examples based on existing minority samples



[1] SMOTE: synthetic minority over-sampling technique NV Chawla, KW Bowyer, LO Hall, WP Kegelmeyer. Journal of artificial intelligence research, 2002

Oversampling: SMOTE

- SMOTE = Synthetic Minority Over-sampling Technique)
- It creates new minority examples by interpolating between nearby minority points in feature space
- It creates new points <u>between existing minority samples</u>, expanding the region of the feature space occupied by the minority class

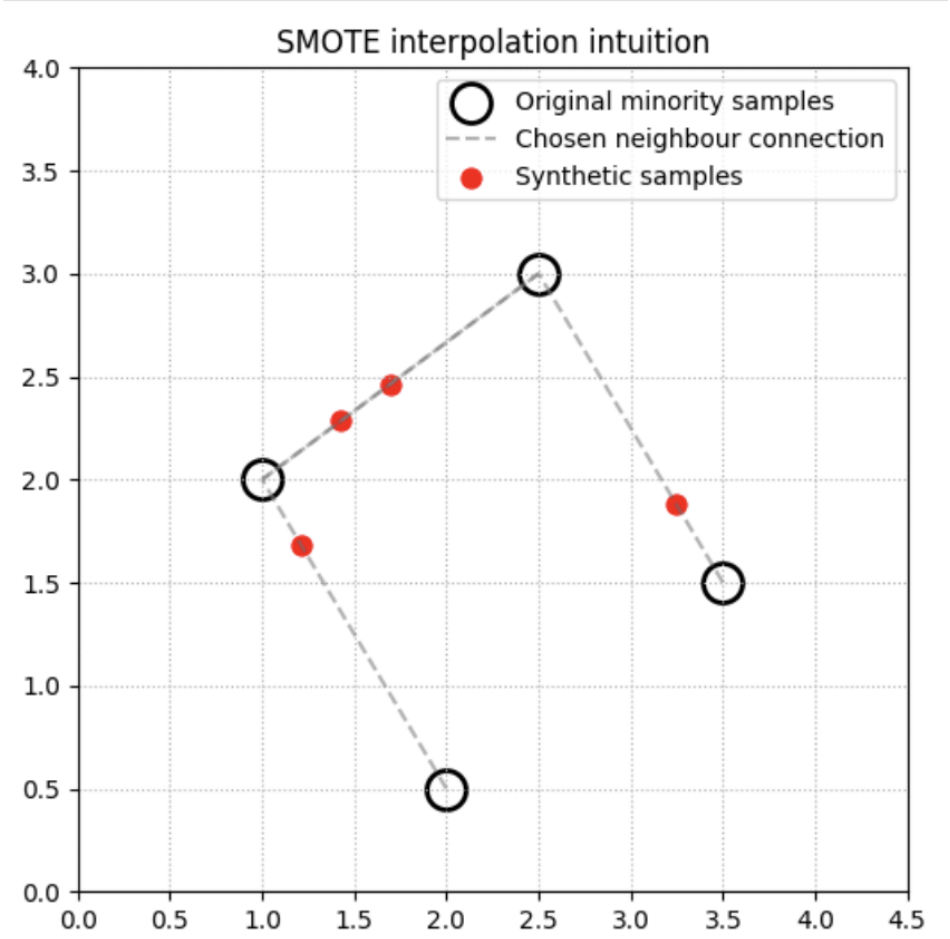
[1] SMOTE: synthetic minority over-sampling technique NV Chawla, KW Bowyer, LO Hall, WP Kegelmeyer. Journal of artificial intelligence research, 2002

Oversampling: SMOTE (intuition)

1. Choose a minority instance
Randomly pick a sample from the minority class.

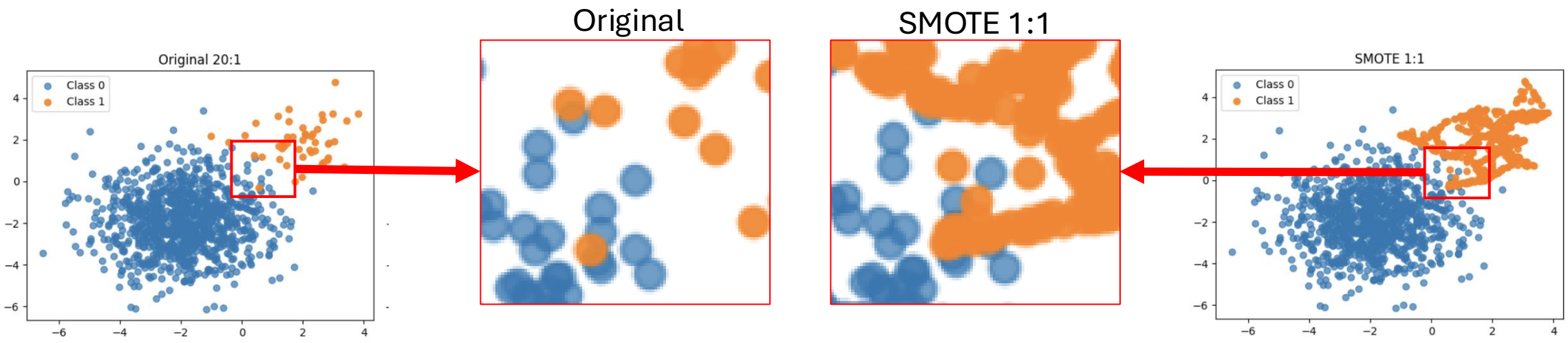
2. Find a nearby minority neighbour
Identify one of its *k nearest neighbours* from the minority class.

3. Create a synthetic point between them
Generate a new instance somewhere along the line segment connecting the two points.



[1] SMOTE: synthetic minority over-sampling technique NV Chawla, KW Bowyer, LO Hall, WP Kegelmeyer. Journal of artificial intelligence research, 2002

Oversampling: SMOTE limitations



- Creates synthetic instances that are not real, just interpolations between existing minority samples
- That becomes a problem when those synthetic points are **not plausible** or **do not reflect the true data distribution**

Cost-sensitive (Class weighting)

- Assign different importance (weights) to classes during training
- Misclassifying minority class instances incurs a higher penalty
- Encourages the model to pay more attention to rare classes

Example: (Balanced) Decision trees

- Class weights modify how split quality is evaluated during tree construction
- Minority class instances have greater influence on the splits chosen during tree construction

Cost-sensitive: *(Balanced)* Decision Trees*

1. **Start with class weights**
   Example: majority class weight = 1, minority class weight = 20.
2. **For each candidate split, count weighted class frequencies**
   Instead of counting each sample as 1, count it according to its class weight.
3. **Compute impurity using weighted counts**
   Use the weighted class proportions to compute entropy or Gini.
4. **Compare candidate splits as usual**
   The tree still chooses the split with the best impurity reduction.

*** Inside a Random Forest or any other ensemble:** repeat this for every tree.

Cost-sensitive: *(Balanced)* Decision Trees*

1. **Start with class weights**
   Example: majority class weight = 1, minority class weight = 20.
2. **For each candidate split, count weighted class frequencies**
   Instead of counting each sample as 1, count it according to its class weight.
3. **Compute impurity using weighted counts**
   Use the weighted class proportions to compute entropy or Gini.
4. **Compare candidate splits as usual**
   The tree still chooses the split with the best impurity reduction.

Class weighting **does not change the labels or create new data**. It changes how much each example counts when evaluating splits.

*** Inside a Random Forest or any other ensemble:** repeat this for every tree.

Dimensionality Reduction

From Feature Selection to Dimensionality Reduction

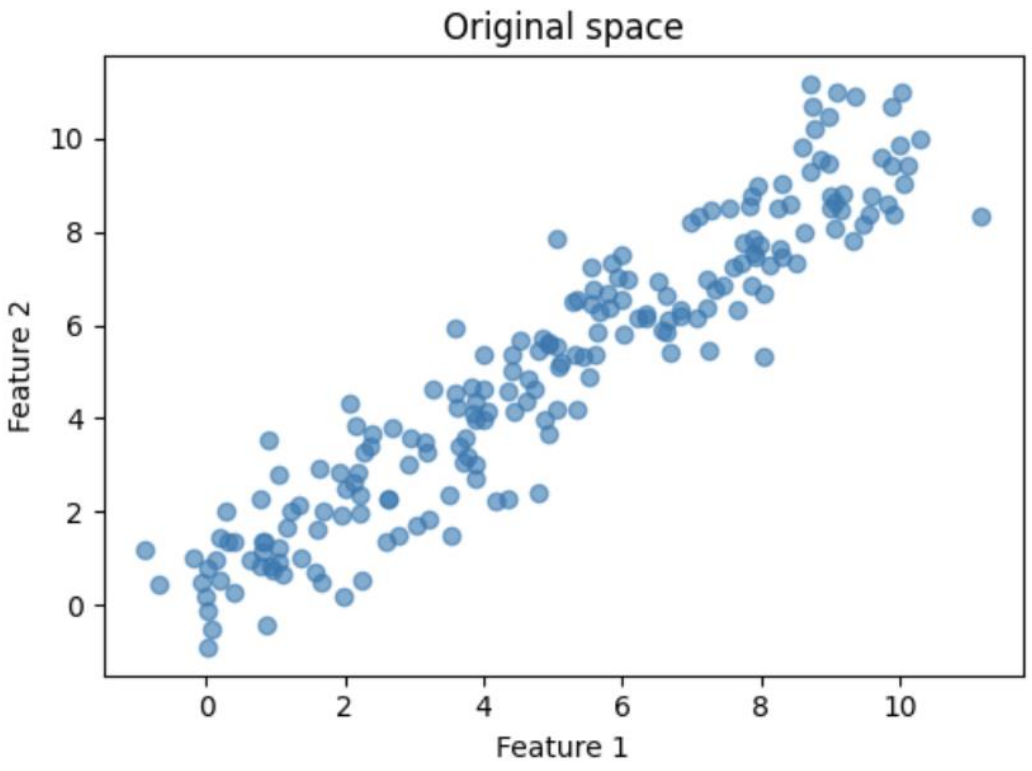
- **Feature selection**
  - keep a subset of the original features
  - removes irrelevant or redundant features
  - improves efficiency, simplicity, and sometimes performance
  - assumes that useful features already exist

What if the useful information is spread across multiple features?

- **Dimensionality reduction**
  - transforms the feature space into fewer variables
  - creates new features (less interpretable)
  - can capture hidden structure in the data

Example: Hidden structure in the data

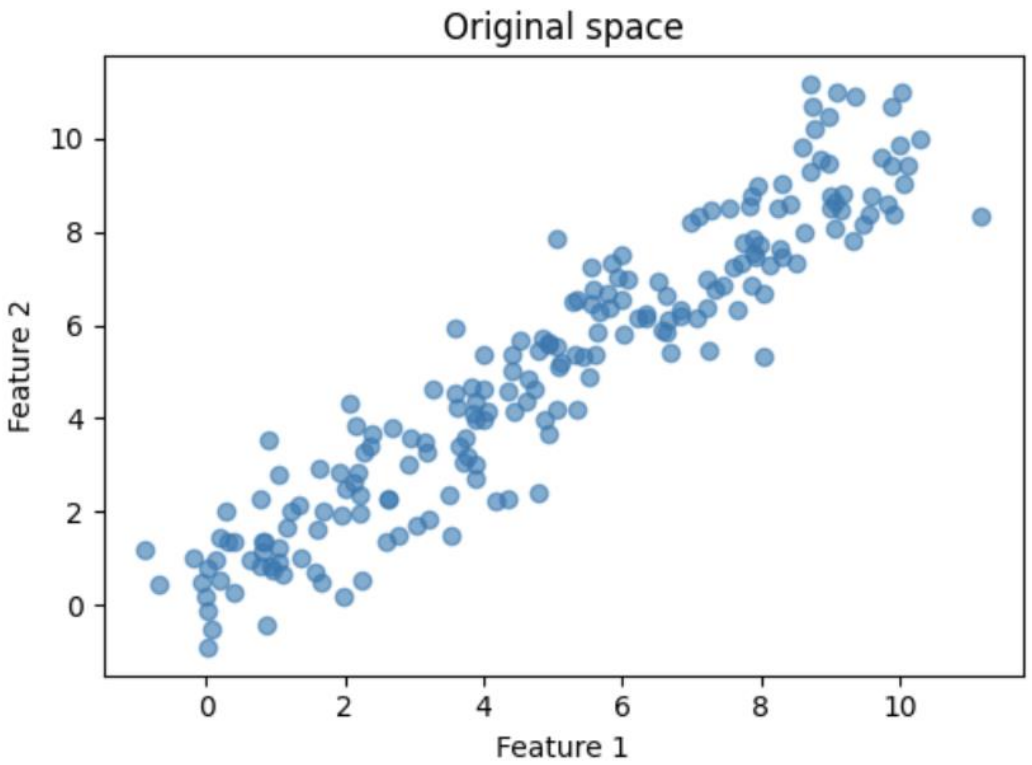
Assume two features...

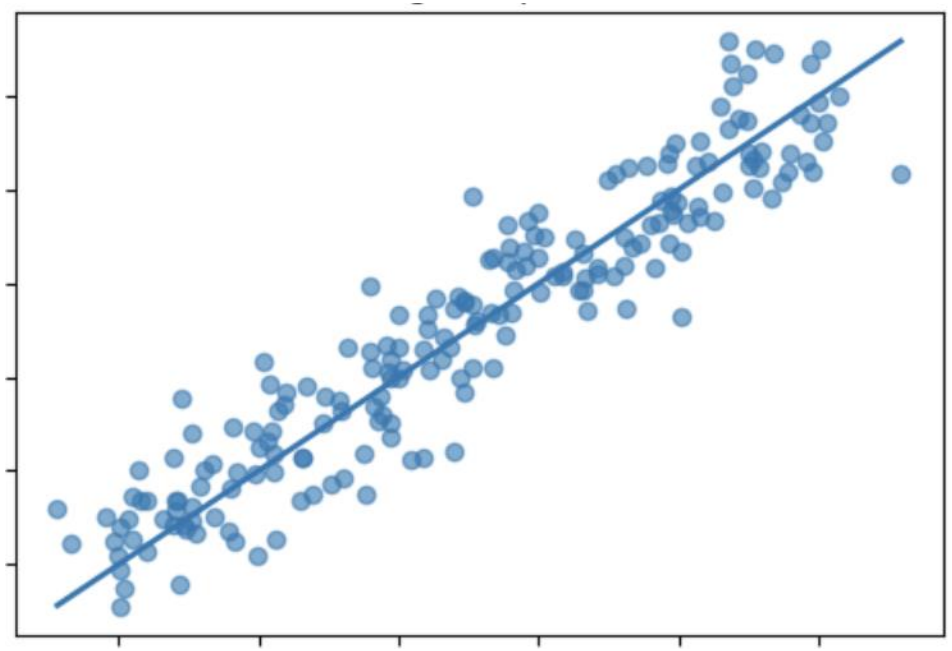


The data looks 2D...

Example: Hidden structure in the data

Assume two features...



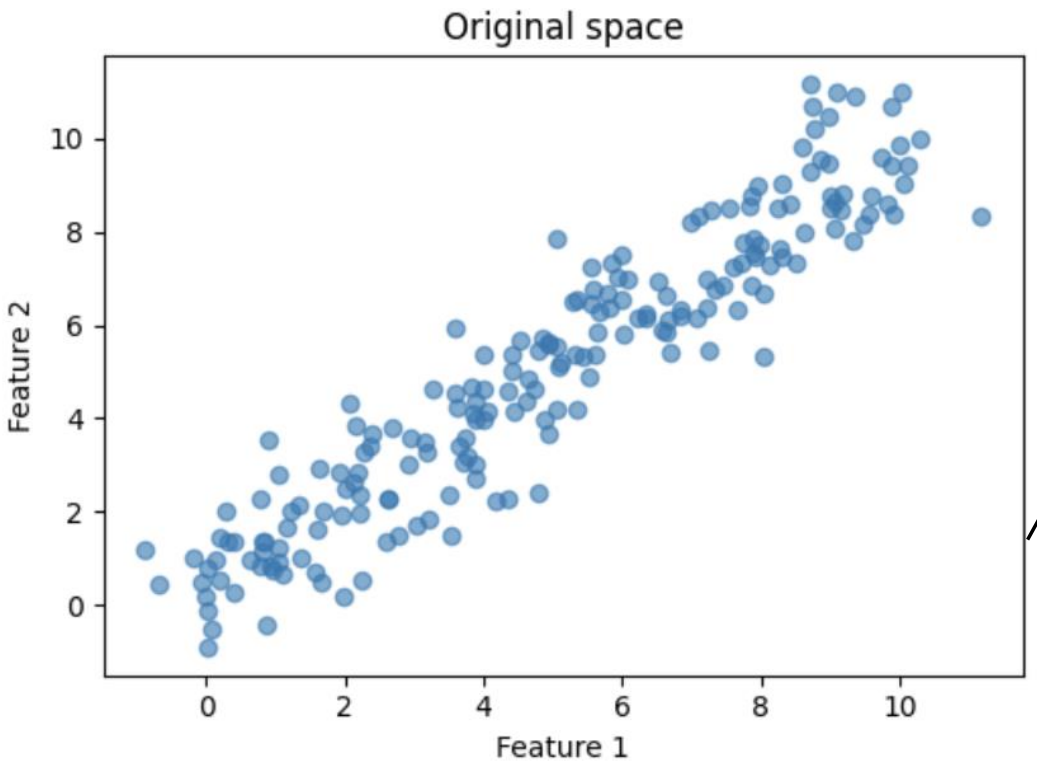


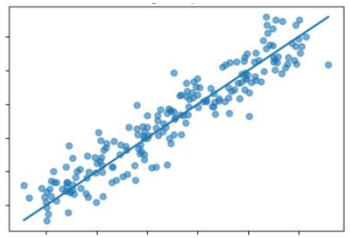
The data looks 2D...

but follows a clear pattern

Example: Hidden structure in the data

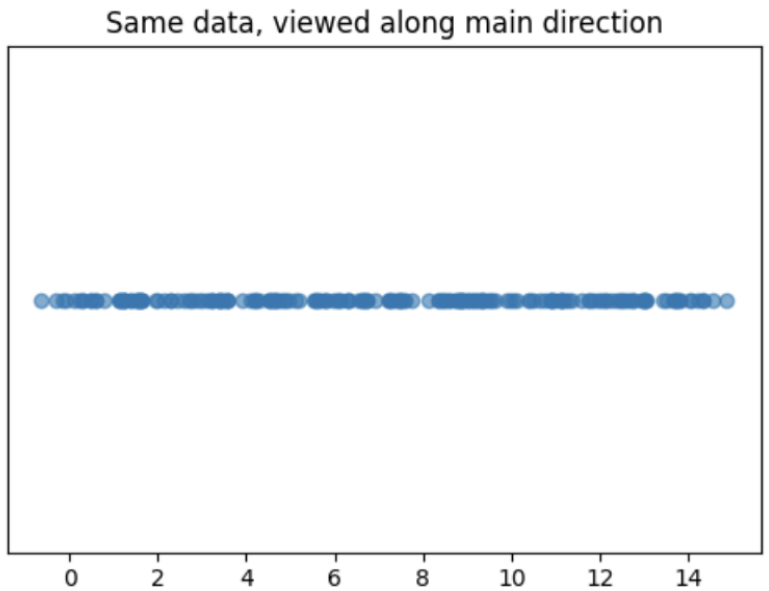
Assume two features...





but follows a clear pattern

New combined feature



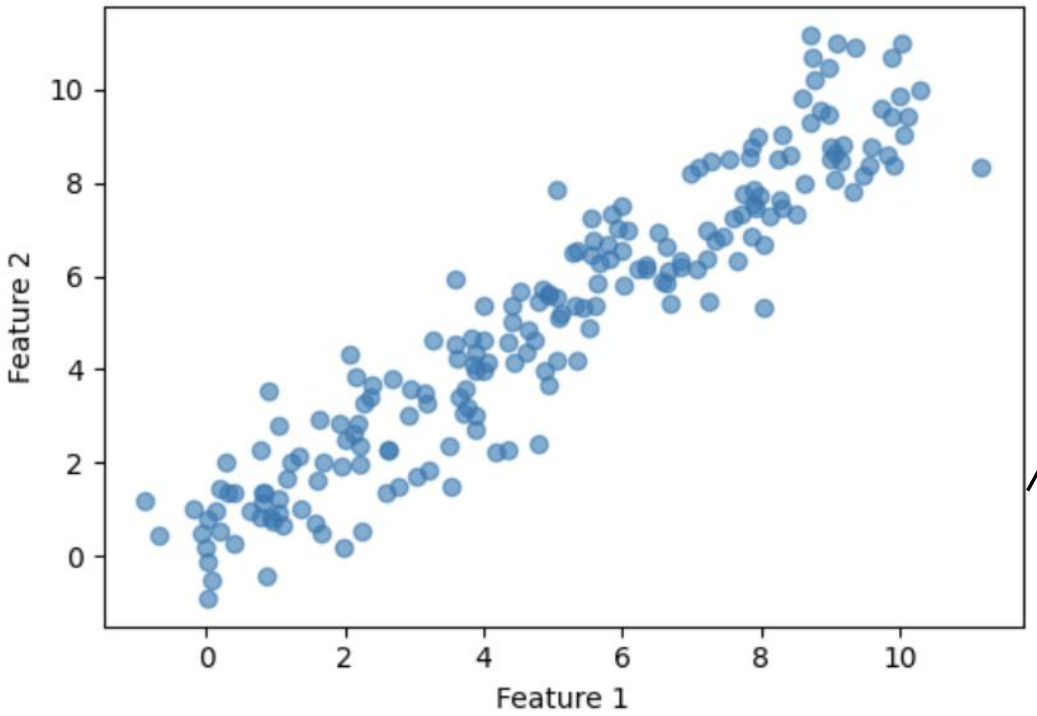
The data looks 2D...

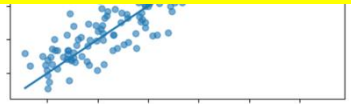
The important information is not aligned with individual features, it lies along a direction across both features.

Example: Hidden structure in the data

Assum

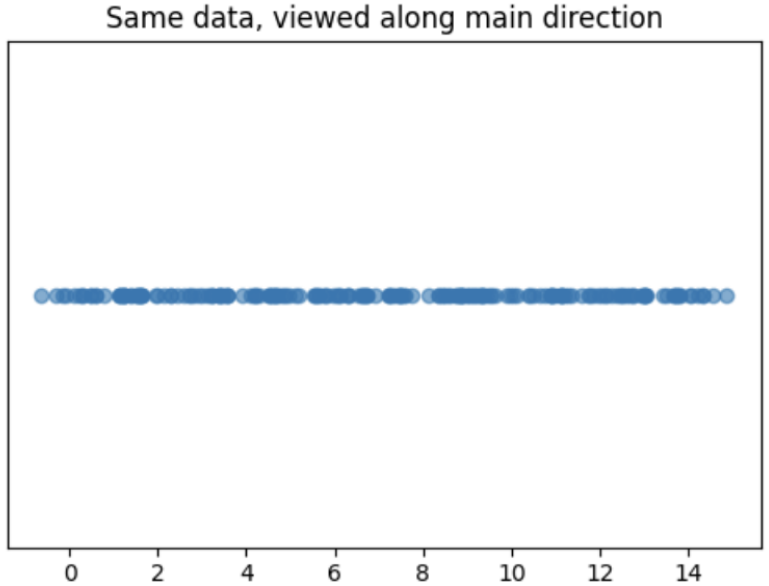
IMPORTANT: We are not creating new data, we are transforming how the data is represented





but follows a clear pattern

New combined feature



The data looks 2D...

The important information is not aligned with individual features, it lies along a direction across both features.

Dimensionality Reduction

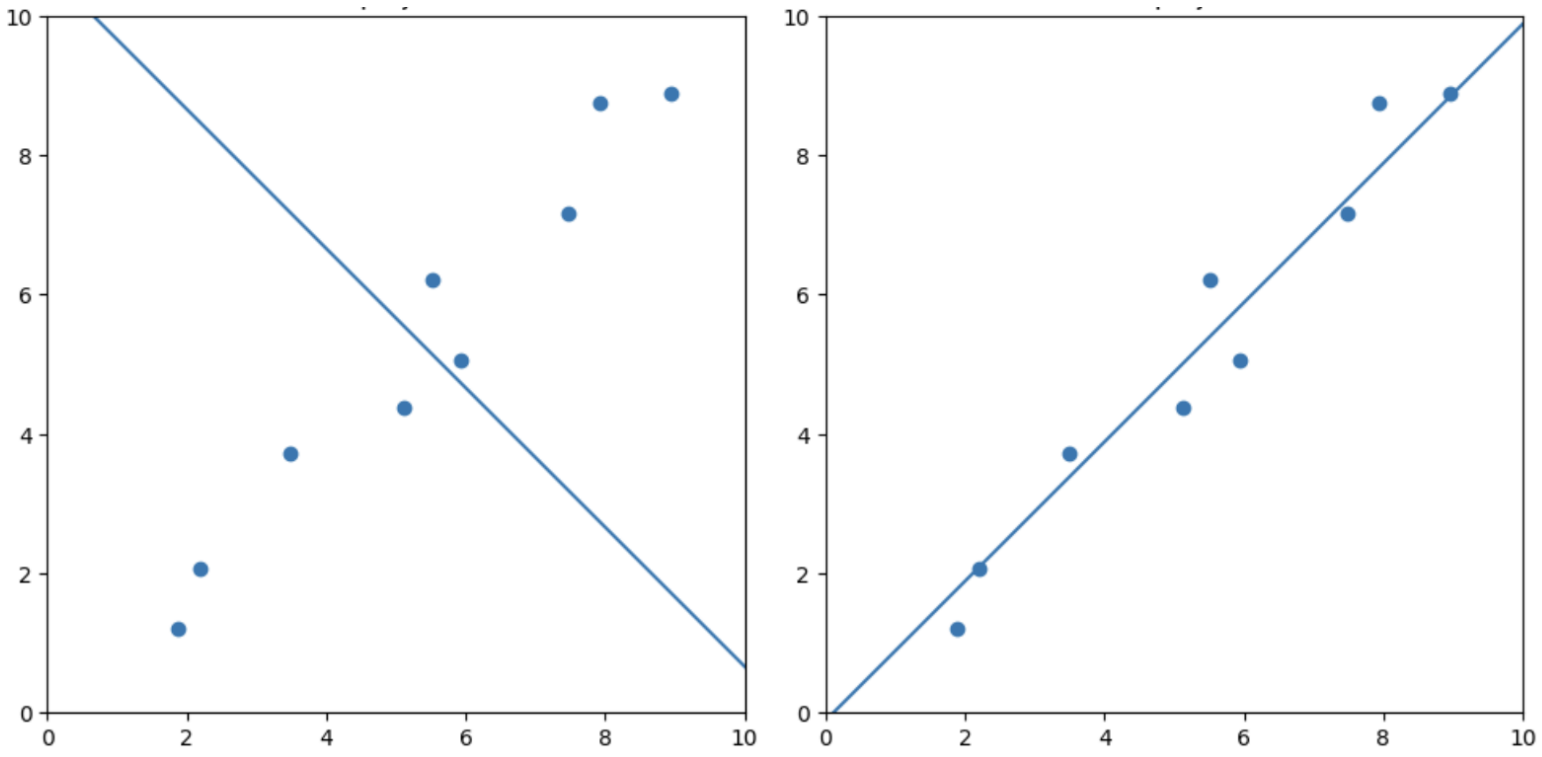
- Transform data into a lower-dimensional space
- Keep as much important information as possible
- Reduce noise and redundancy

- **Different methods:**
  - **PCA** (Principal Component Analysis): captures directions of maximum variation (unsupervised)
  - **LDA** (Linear Discriminant Analysis): focuses on separating classes (supervised)
  - **ICA** (Independent Component Analysis): finds independent underlying signals
  - **Autoencoders**: learn representations from data (neural networks)

PCA - Intuition

We want to represent these points using a line.

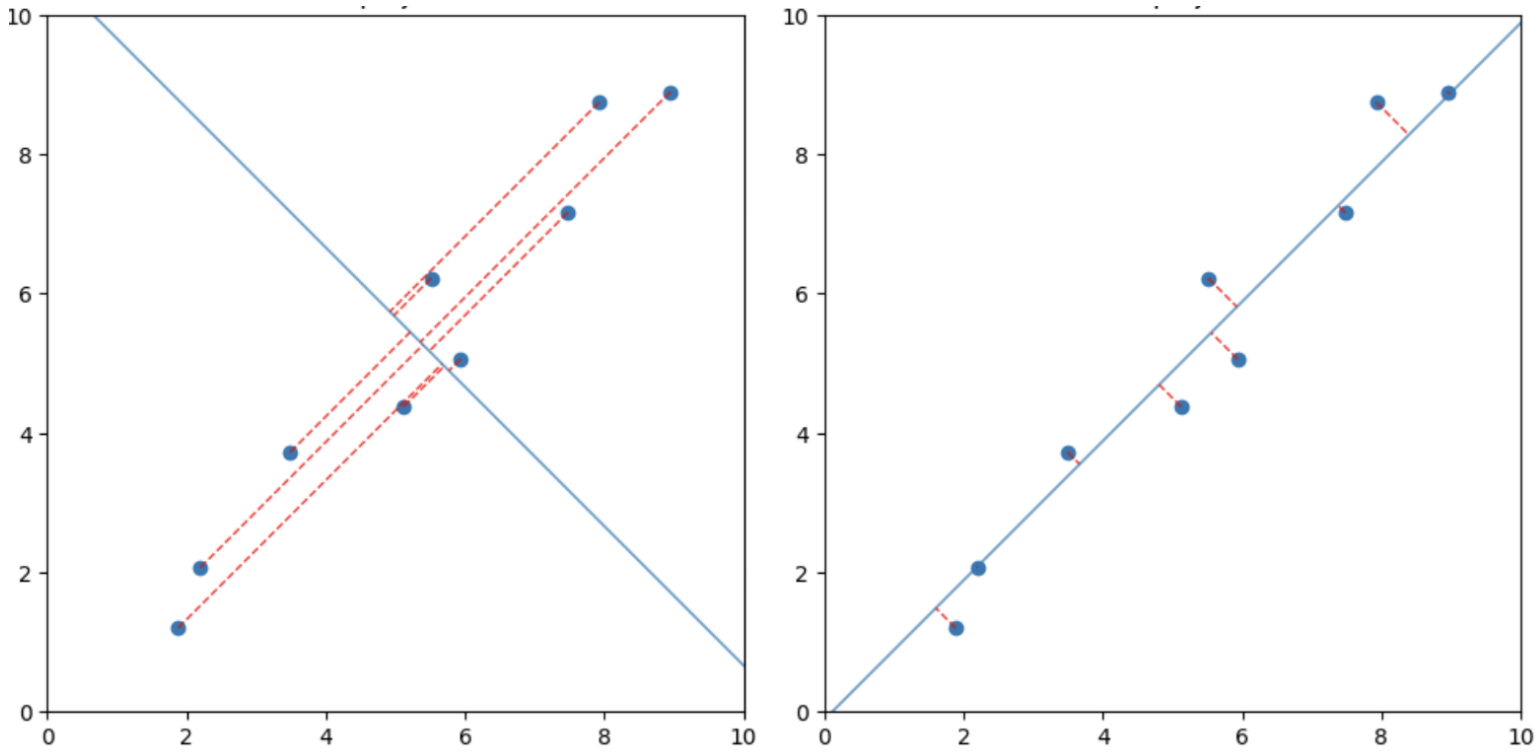
Which line preserves the structure better?



PCA - Intuition

We want to represent these points using a line.

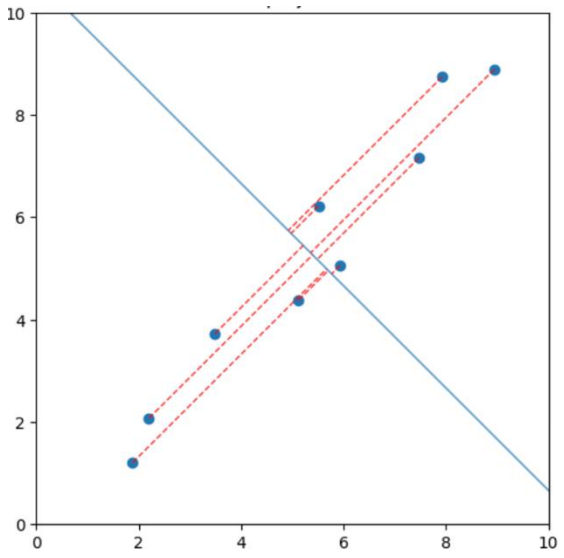
Which line preserves the structure better?

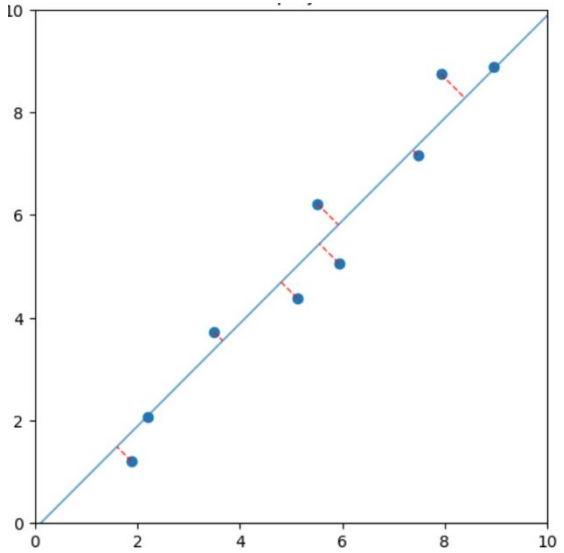


These red dashed lines show how much information we lose when we project onto the line

PCA

- PCA finds directions where the data varies the most (maximising how spread out the data is)
- Equivalently, it chooses the direction that makes the projection error* as small as possible (minimising how far points are from the line)
- In practice, PCA:
  - Identifies directions** that capture the most variation
  - Projects the data onto those directions
- We then keep $d < n$ directions (dimensionality reduction), where n is the number of original dimensions (features)





* Those distances (red dashed lines) measured perpendicular to the line

** These are called principal components

PCA – High-level procedure

- PCA identifies the direction with the **maximum variation**
  - This is the **first principal component (PC_1)**
  - The best one-dimensional (1D) summary of the data

- PCA also identifies a second principal component (PC_2)*
  - Captures remaining variation
  - PC_2 is orthogonal to PC_1 (Why?)

- This continues, producing up to ***n*** PCs (same as original number of features)
  - Then we choose how many we want to keep
  - Each component is chosen to explain what the previous ones didn't

(What happens if we keep all n PCs?)

PCA – Intuition about *PCs*

The more components we keep, the better we represent the original data

- PC_1 → most important variation
- PC_2 → next most important variation
- PC_3 → finer variation

…

- PC_n → smallest remaining variation

PCA – Intuition about *PCs*

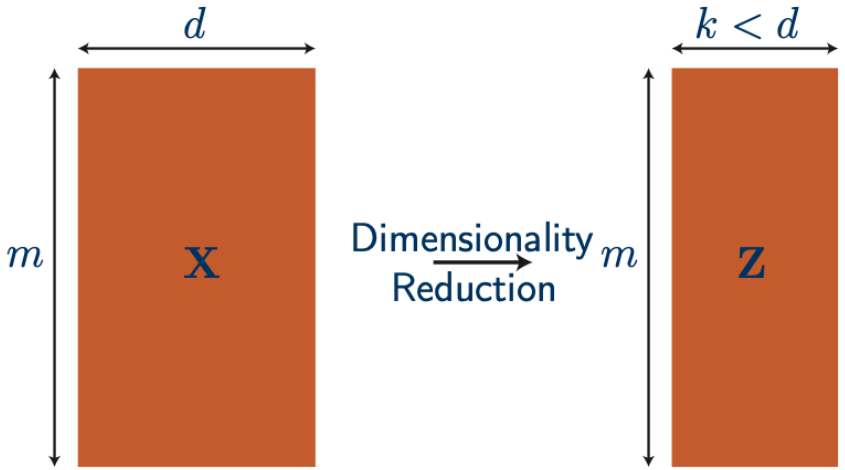
The more components we keep, the better we represent the original data

- PC_1 → most important variation
- PC_2 → next most important variation
- PC_3 → finer variation

…

- PC_n → smallest remaining variation

Just to emphasize that we do not alter the number of examples in the data

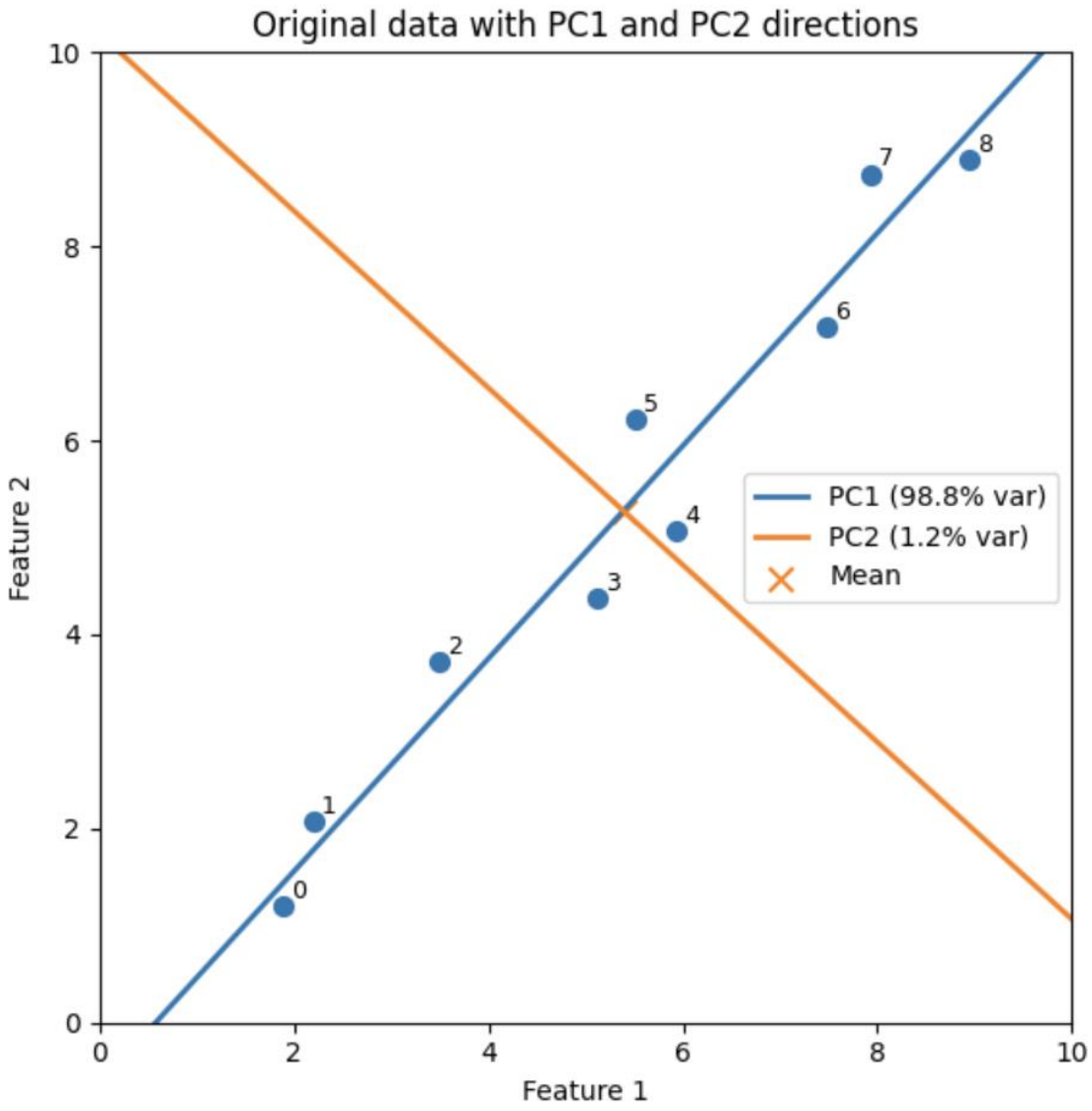


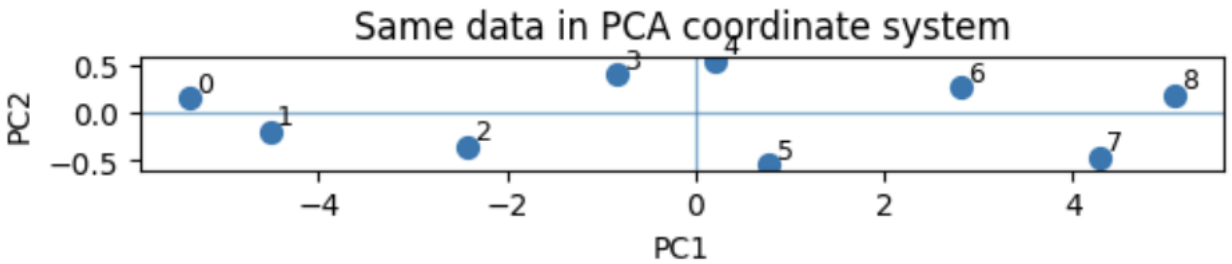
PCA – Example

- Same data as before (2 features)
- Showing PC1 and PC2
- PC1 explains almost all the variation
- X is the centre of the data, PCA rotates the axes around this point

Instance	Feature1	Feature2
0	1.882	1.205
1	2.2	2.072
2	3.489	3.727
3	5.12	4.381
4	5.934	5.061
5	5.511	6.222
6	7.475	7.167
7	7.924	8.747
8	8.948	8.897

PC1	PC2
-5.369	0.154
-4.515	-0.196
-2.423	-0.359
-0.841	0.406
0.21	0.548
0.783	-0.546
2.804	0.268
4.275	-0.465
5.076	0.19





PCA – Implementation

- We compute PCA by **maximising variance**; equivalently, this **minimises reconstruction error**

- In practice, PCA is implemented using linear algebra:
  - **Covariance matrix**: captures how features vary together
  - **Eigenvectors**: give the principal components (PCs)

- Singular Value Decomposition (SVD)
  - efficient way to compute PCA
  - avoids explicitly computing the covariance matrix

Wrap-up

- Feature selection
  - Filter, wrappers and embedding
  - Not so popular anymore, still useful if the goal is interpretability
- Evaluation
  - K-fold CV: more confidence in the performance estimation on unseen data
- Class imbalanced
  - We can address as a preprocessing step (under/oversampling)
  - Weighting is preferred instead of oversampling to avoid “unrealistic” instances
- Dimensionality reduction
  - PCA

Next lecture: Tutorial